FantaWorld: A Microservices-Based E-Commerce

Matteo Bollecchino 7198883

Contents

1 Introduction

The purpose of this project is the design and implementation of a scalable, fault-tolerant, and decentralized e-commerce platform built on top of a microservices architecture. In modern software engineering, moving away from monolithic systems towards decoupled services represents a critical paradigm to guarantee independent scalability, high availability, and loose coupling.

The system simulates a complete online store for fantasy and sci-fi based items (i.e., action figures, books, comics, mangas, etc.) where users interact through a responsive web interface. To guarantee proper access control and operations separation, the application defines two distinct roles:

- **User:** A standard authenticated user who can browse the product catalog, manage items inside their virtual shopping cart (adding, updating quantities, or removing products), place orders, inspect its orders.
- **Admin:** A privileged user who inherits all user functionalities and is granted exclusive access to administrative capabilities. These include listing registered users, adding new products to the catalog, updating catalog properties (prices and stock availability), deleting items, listing all the orders of a specific user and changing the operational status of that placed orders (*Processing*, *Shipped*, *Delivered*, *Cancelled*).

Crucially, to prevent state inconsistencies and restrict the scope to a robust deterministic environment, user roles are immutable from the application's runtime perspective (i.e., a standard client cannot be promoted to an administrator during the session). The only administrative accounts available are default administrative credentials provisioned automatically when the Authentication Service boots. For this implementation of an e-commerce site, both users and admins will be uniquely identified by a username and a password.

2 E-Commerce Structure and System Architecture

The system layout is strictly decentralized and follows the *Database-per-Service* architectural pattern. The system is composed of an API Web Server acting as a central Orchestrator, alongside five independent microservices communicating via gRPC.

2.1 Microservices Breakdown

Each microservice is an isolated process with exclusive access to its own relational database instance, completely preventing direct data-layer coupling:

1. **Authentication (Auth) Service:** Responsible for user registration, secure password hashing, credential verification, and user metadata retrieval.
2. **Catalog Service:** Manages the inventory data, providing gRPC endpoints to retrieve single items, list the entire catalog, or perform administrative mutations (insertions, remotions, price and stock updates).
3. **Cart Service:** Maintains the temporary state of customers' shopping baskets. It handles items accumulation, quantity updates, and total price calculations.
4. **Order Service:** Records finalized purchases, tracks the transactional history, assigns unique identifiers, and manages state machine transitions for each order.
5. **Payment Service:** Encapsulates a simplified, deterministic payment gateway that tracks payment attempts, matches them against order identifiers, and ensures cryptographic financial records are persisted.

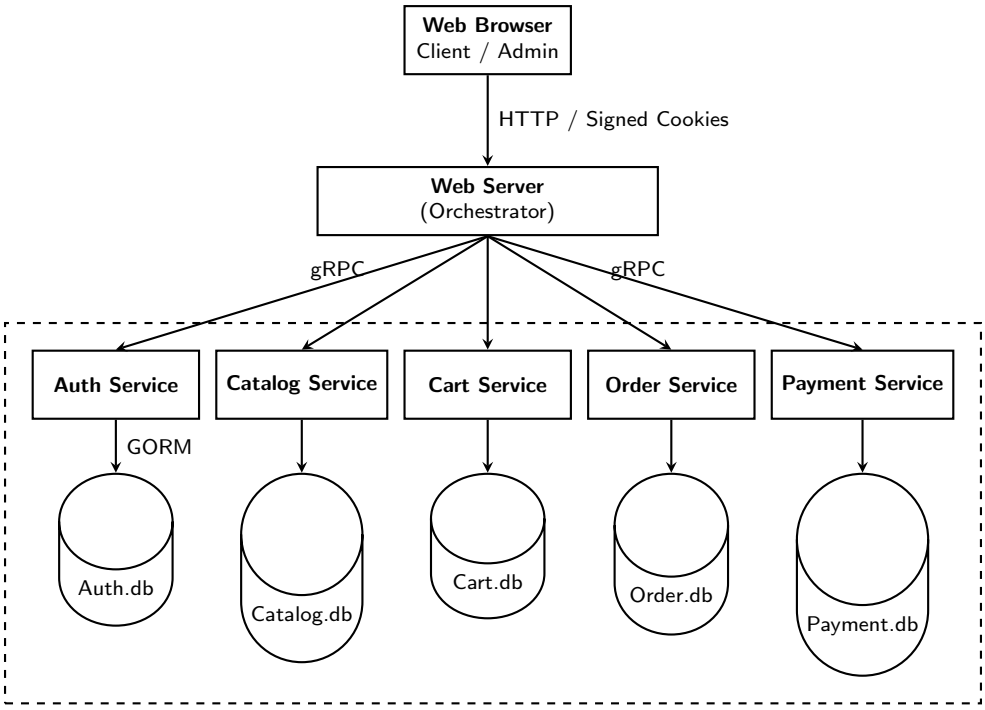


Figure 1: System Architecture Diagram

2.2 The Web Server as an Orchestrator

The Web Server represents the unique entry point for web clients. It serves static assets, renders HTML templates, and orchestrates backend interactions. Rather than allowing messy peer-to-peer communication among backend services, the Web Server implements a synchronous orchestration logic: it coordinates multi-step workflows (e.g., transforming a cart into an order and proceeding to payment) by dispatching linear gRPC requests to the underlying cluster, simplifying error tracking and system observability.

Administrative actions have corresponding UI controls restricted at the HTML layer via template conditions (rendering buttons exclusively for administrative roles). Role validation is enforced within the handler methods inside `ecommerce/web/server/main.go`. These checks are omitted from the backend microservices to maintain a strict separation of concerns; backend services remain lightweight and decoupled from HTTP protocol specificities, cookie parameters, or session lifecycles.

In this architecture, the backend gRPC microservices do not check user roles or sessions directly; instead, they operate under a principle of **Blind Trust**. In a real deployment, these services are hidden inside a private network and cannot be accessed from the internet. The Web Server acts as the only entry point. Because a request can only reach the backend after the Web Server verifies the user's login and role, the microservices safely assume that every incoming gRPC call is already authorized. This choice avoids checking security tokens at every internal step, making the network communication much faster.

2.3 gRPC Interface and Protocol Buffer Conventions

To streamline network error handling across the gRPC boundaries, all Protocol Buffer (`.proto`) response messages are structured using a uniform convention: they expose a `error_message` string field. If this field evaluates to empty or nil, the web orchestration layer can interpret the RPC request as successful. Conversely, any non-empty string indicates a specific application error that must be handled by the orchestrator.

3 Design Choices

Every component of this e-commerce platform has been engineered adhering to specific technical trade-offs, prioritizing performance, ease of deployment, and adherence to programming best practices.

3.1 Service Abstraction and Component Decoupling

The architecture enforces a strict structural organization repeated identically across every microservice within the `internal` directory. Both the service Server and its concrete Repository implement a shared interface abstraction (e.g., `AuthServiceInterface`). The Server acts as the high-level boundary responding to network events, while the Repository operates directly on the data storage tier. The actual database connection instance is initialized exclusively within the service's `main.go` entry point and passed downward via *Dependency Injection*.

Input verification checks are intentionally duplicated at both the Server and Repository levels. While input sanitization is typically expected primarily at the server layer, this design choice satisfies three key architectural goals:

- **Component Decoupling:** It insulates the Server layer from the concrete underlying storage implementation details.
- **Active Repositories:** It prevents the Repository from becoming a "dummy" or passive data-access wrapper, ensuring data layer invariants are validated locally.
- **Testability:** Input validation constraints become highly decoupled and far easier to isolate through unit testing frameworks.

3.2 Authentication, Administrative Seeding, and Hashing

Usernames must be globally unique across the entire platform, regardless of whether the account belongs to a standard user or an administrator. All user passwords are encrypted within the Authentication Service using the `bcrypt` hashing package before being persisted.

To initialize the environment, the platform automatically provisions the following default accounts upon booting the Authentication Service:

- **Default Clients:** Marco (`DefaultPassword1+`), Noemi (`DefaultPassword2+`).
- **Default Admins:** adminBolle (`AdminPassword1+`), adminDani (`AdminPassword2+`).

The execution of administrative seeding leverages an internal repository method named `CreateAdmin`. Crucially, `CreateAdmin` is intentionally excluded from the public `AuthServiceInterface` contract. Because administrative generation is strictly an internal setup operation, exposing this method through the public interface would force its inclusion within the public `.proto` files. This exposure would erroneously allow external services to trigger administrative creation over the network. Although `CreateAdmin` shares structural similarities with the public `Register` method, this configuration does not constitute redundant code duplication, as the two methods serve entirely distinct operational layers and access control scopes.

3.3 Catalog and Inventory Semantics

The Catalog Service permits items to hold a total available stock quantity of **0** (representing an OUT OF STOCK status) or a price point of **0** (indicating that the item is entirely FREE). The underlying repository allows the creation of multiple `CatalogItem` entries sharing identical attributes but possessing unique primary identifiers; checking for duplicate catalog data entries remains an operational responsibility of the administrator.

3.4 Persistent File SQLite Databases vs. In-Memory State

To support real-world distributed semantics, the idea of utilising volatile in-memory databases (`sqlite.Open(":memory:")`) has been replaced by using persistent ones, localized file-based SQLite databases (`.db`).

This choice offers significant architectural benefits. First, it ensures **state durability**, protecting data against unexpected microservice crashes or restarts. Second, it perfectly honors the **Database-per-Service** paradigm: each service encapsulates its own `.db` file, preventing cross-service table joins and enforcing strict data isolation. Lastly, it offers **zero-configuration deployment**: the system requires no external database server installations, as the GORM Object-Relational Mapper automatically provisions files and executes schema migrations (*AutoMigrate*) at boot time.

3.5 Cart Mechanics and GORM Relation Management

Products can only be added to a cart if the user has successfully established an active authenticated session. Cart items can accommodate a price of **0** for free items, but they must maintain a minimum quantity of at least **1**. Because a complete financial transaction component is simulated, cart items do not strictly require independent price attributes at rest, as totals can be derived directly from the catalog state.

When interacting with GORM across related relational schemas, utilizing the `Preload` mechanism is mandatory to explicitly eager-load associations during standard queries:

```
r.db.Preload("Items").Where("username = ?", username).First(&cart).Error
```

Failing to invoke `Preload` causes GORM to leave relation slices empty or nil. Similarly, to persist complex nested structs and relational changes successfully, database operations must enforce association saving rules via custom GORM session blocks:

```
r.db.Session(&gorm.Session{FullSaveAssociations: true}).Create(cart1).Error
r.db.Session(&gorm.Session{FullSaveAssociations: true}).Save(cart).Error
```

Notably, inside the `RemoveItemFromCart` in the correspondent repository, item deletion is executed via a direct SQL query targeting the specific table rather than using GORM's automated Association abstraction handlers. When executing an association clearance command, GORM attempts to run an `UPDATE` statement that resets the foreign key field (`cart_username`) to `NULL`. However, the domain model defines `cart_username` within `CartItem.go` with a strict `NOT NULL` database constraint, meaning GORM's default behavior would trigger constraint violation errors. Direct table deletion bypasses this limitation cleanly.

The architecture implements a **Strict Session** model for cart cleanup: as soon as a payment transaction concludes successfully, the system calls a `ClearCart` routine, performing a complete *tabula rasa* of the user's basket. This forces a clean state for future user interactions, avoiding the overhead of managing stale or leftover items across subsequent sessions.

3.6 Order Lifecycle, Custom UUIDs, and Payment States

When a checkout workflow begins, unique order identifiers are generated as strings using Universally Unique Lexicographically Sortable Identifiers (**ULID**):

```
orderID := ulid.Make().String()
```

Beyond enforcing the absolute uniqueness of the generated `orderID`, the platform executes no deep structural validation regarding the individual contents of an order. This design accommodates cases where a user wishes to place multiple distinct orders containing identical sets of items. An order total is permitted to equal **0** if all underlying items are free. To maintain an authentic presentation layer within `web/server/main.go`, floating-point total calculations are formatted using truncation routines to eliminate long, unrealistic decimal trailing digits:

```
math.Trunc(priceRes.GetTotalPrice()*100)/100
```

The system handles total calculations differently depending on the HTTP verb context:

- **In `orderHandler` (GET):** The system avoids calling `GetOrderPrice` because the order has not yet been initialized in the database. Instead, it utilizes `CalculateTotalPrice`, of the cart interface, to render a provisional total directly on the user's checkout summary page.
- **In `paymentHandler` (POST):** To ensure maximum precision from the microservices layer, the orchestrator invokes `CreateOrder`, captures the real generated tracking identifier, and immediately feeds it into the `GetOrderPrice` microservice routine. If `CreateOrder` encounters an unrecoverable system fault, it returns an empty string.

Once an order is established, it initiates in a **PENDING** state. Following successful payment confirmation, the order transitions automatically to a **PROCESSING** state. Further updates to advanced life states (**SHIPPED**, **DELIVERED**, **CANCELLED**) are restricted and can only be executed manually by admins.

The actual physical depletion of product stock inside the Catalog database occurs exclusively after a payment transaction returns a successful status, as this marks the exact moment legal product ownership is transferred to the buyer. While this delayed deduction policy can theoretically introduce brief window inconsistencies for highly concurrent purchases of the same product, it represents a conscious design trade-off: managing concurrent overselling or inventory exhaustion is delegated to the platform's operational warehouse fulfillment workflows rather than locking database transactions during checkout.

3.7 Idiomatic Routing via Go Standard Library

To avoid bloating the project with heavy external web frameworks, the HTTP routing layer within `web/main.go` is entirely designed using the native Go standard library (`http.ServeMux`).

In accordance with idiomatic Go development, HTTP method filtering (GET vs. POST) is handled directly inside the custom handlers using standard conditional statements (e.g., `if r.Method == http.MethodPost`). This decision drastically improves *code cohesion* for form-centric endpoints. For instance, the handler rendering a form (GET) and processing its submission (POST) coexists within a single functional unit. This structure simplifies state context sharing, inputs validation, and the immediate re-rendering of UI templates when errors occur.

3.8 Session Management via Signed Cookies

User sessions are tracked via the `gorilla/sessions` package, which integrates directly with `gorilla/securecookie`. Upon successful authentication, the Web Server injects three explicit key-value pairs into the session store: `username`, `role`, and `logged_in`.

By checking these values locally, the Web Server eliminates the network overhead of executing a gRPC call to the Auth Service on every page load. Security is guaranteed by passing a secure cryptographic secret key to the `CookieStore`: cookies transmitted to the browser are digitally signed, ensuring that any malicious user attempt to forge session attributes (e.g., modifying the role string to escalate privileges) automatically invalidates the signature, causing the system to reject the request.

For local deployment and ease of examination, both secret keys required `sessions.NewCookieStore` are defined locally within the codebase, allowing the server to execute immediately without external configuration steps. While hardcoding these keys ensures simplicity during development, it represents a security vulnerability for production environments. In a real-world scenario, these keys would be safely injected at runtime through environment variables, keeping them excluded from source control tracking.

During testing, when a user opens a new browser tab, the authenticated session state is preserved. This behavior validates that the underlying cookie mechanisms are operating correctly, mirroring a standard production environment. To simulate separate concurrent user interactions simultaneously during evaluations or live demonstrations, developers must utilize distinct browser profiles or isolated incognito sessions.

3.9 Just-In-Time Validation and Order Lifecycle Delimitation

Due to the decoupled nature of distributed databases, a major challenge involves protecting users from purchasing items whose inventory status changed while sitting in the cart. The system addresses this issue by enforcing a strict boundary of responsibility during the checkout pipeline:

- **Pre-Order Validation (Client Responsibility):** Inside the `PaymentHandler`, right before an order is committed, the orchestrator executes synchronous *just-in-time* checks against the Catalog Service. If a product was deleted, its price modified, or its stock exhausted by an admin, the system purges the invalid item from the user's cart and redirects them back to the interface with a `catalog_changed` query flag, forcing a conscious re-evaluation of the basket.
- **Consolidated Transaction (System Responsibility):** The creation of the order via `CreateOrder` acts as the point of no return. Once persisted, the transaction conditions are frozen. Any subsequent inventory or price adjustments belong exclusively to the platform's operational domain. The administrator relies on the `PROCESSING` order state to manually review anomalies or resolve race conditions.

3.10 Payment Idempotency via PRG and State Restrictions

Once a payment record is bound to a specific order identifier, duplicate payment instances are rejected by the database schema. If a payment attempt fails, it can be reprocessed using an updated financial amount. Status modifications on a payment record are only permitted if its current database state evaluates to **PENDING** or **FAILED**. If a payment is already marked as **PAID**, the `ProcessPayment` method (in `paymentServiceRepository.go`) terminates immediately, returning an error to prevent transaction duplication.

Furthermore, to eliminate duplicate payment processing risks triggered by users refreshing their browser or double-clicking form submission elements, the system implements the Post/Redirect/Get (PRG) pattern. The backend handles the payload transaction via an HTTP POST request and then immediately redirects the client to a passive, static GET confirmation page, ensuring subsequent browser reloads do not retrigger the financial workflow.

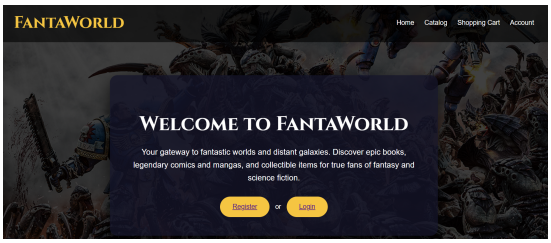
3.11 Transport Credentials for Local Development

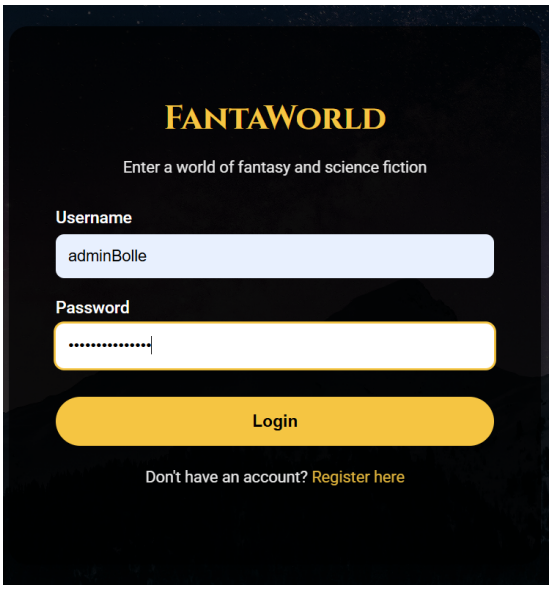
Communication over the gRPC network uses `grpc.WithTransportCredentials(insecure.NewCredentials())`. While production environments necessitate encrypted TLS tunnels to avoid Man-in-the-Middle eavesdropping, local environments running entirely on `localhost` face no external network risks. Disabling TLS local wrappers eliminates the configuration overhead of self-signed certificate generation, allowing to focus entirely on core business logic.

4 Use Case: Making an Order of a New Item in the Catalog

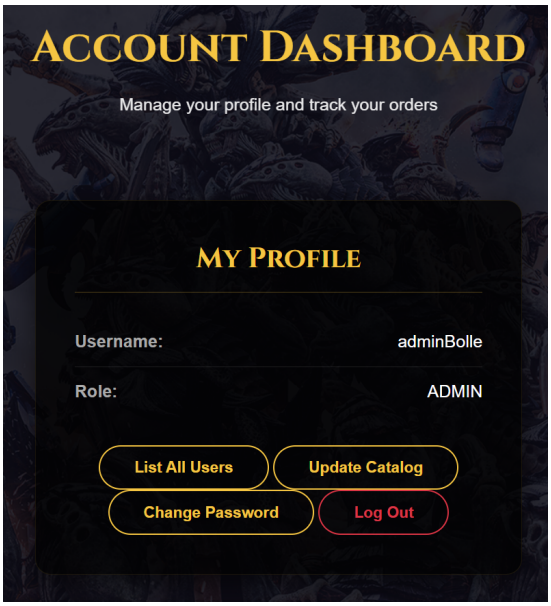
Note: This sequence of screenshots represents the step-by-step user journey and the respective web pages traversed to accomplish the processes that will be described.

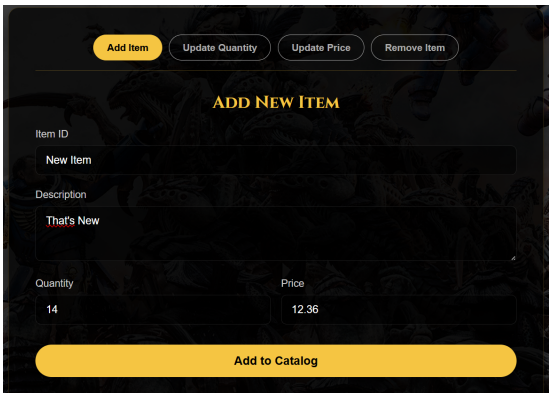
To begin, we authenticate using one of the two pre-configured administrator accounts.



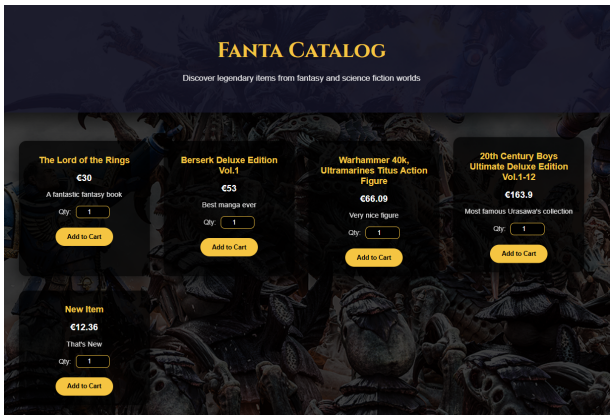


Next, we navigate to the administrator's account page. Clicking the management button redirects us to the catalog update portal. For this specific use case, we select the option to add a new product to the catalog.

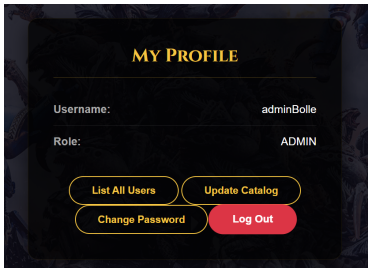


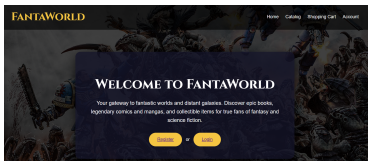


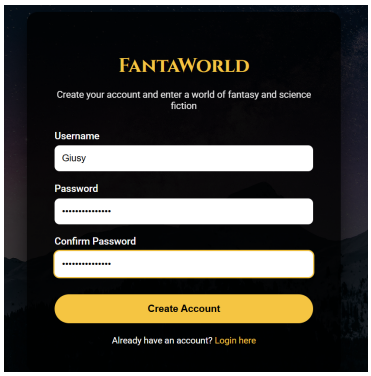
Once the item is successfully added, we can verify that it is visible in the catalog and immediately available for purchase by any user.

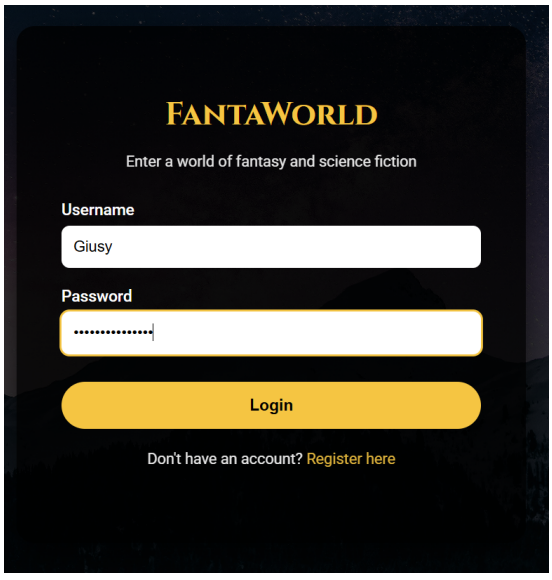


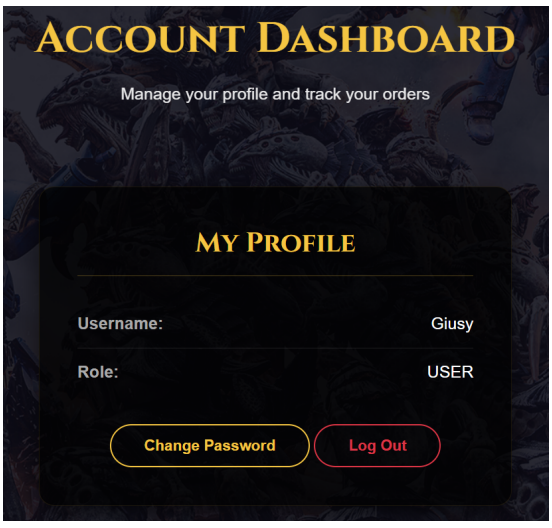
We then log out of the administrator account. After returning to the landing page, we register a new customer account. Upon successful registration, we are redirected to the login page, allowing us to authenticate as the newly created user and proceed with the shopping experience.



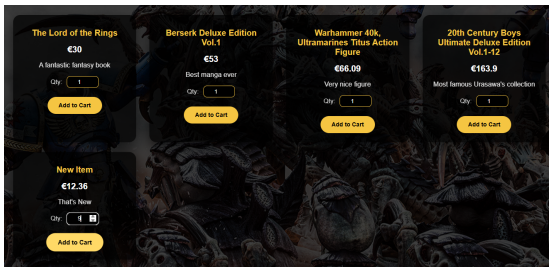


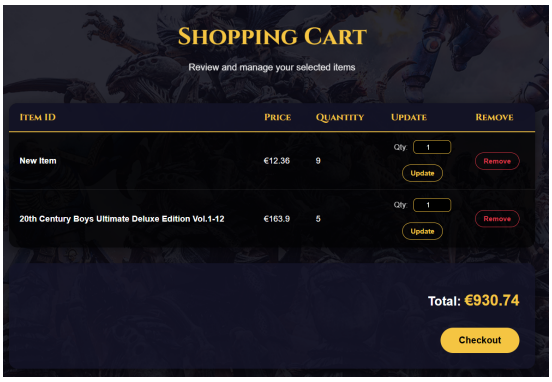




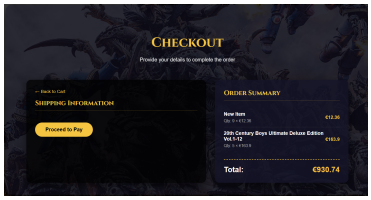


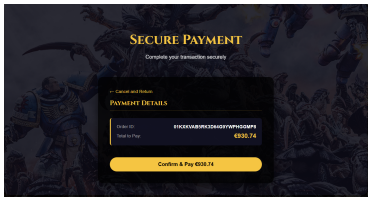
Next, we open the catalog and add both the newly created item and a few pre-existing products to our shopping cart.

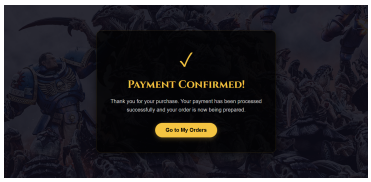




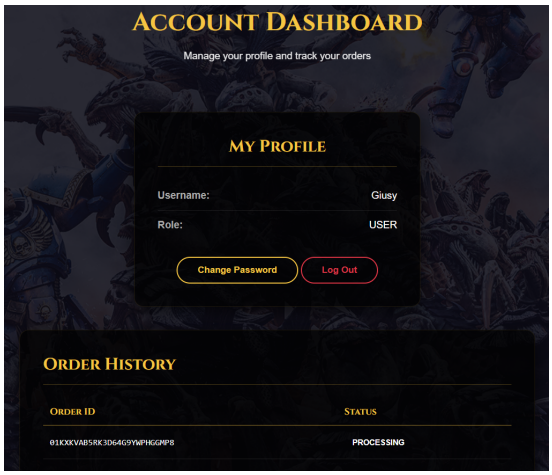
With the items selected, we proceed to the checkout phase to place the order.

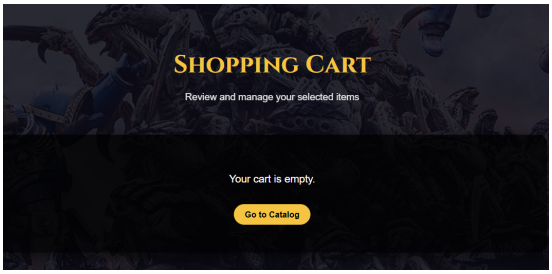






Finally, we can verify that the newly placed order has been successfully processed and is properly listed in the user's personal order history. At the same time, the user's cart has been cleared after the successful elaboration of the order.





5 Strategies for Application Portability

To maximize application portability and ensure the system can be deployed seamlessly across diverse host environments, the platform implements several architectural strategies:

- **Isolated Database-per-Service:** Every microservice encapsulates its data lifecycle within an independent, local SQLite file (`sqlite.Open("service.db")`), removing dependencies on external database servers.
- **Pure-Go Drivers:** The application utilizes a modern, pure-Go SQLite driver that completely eliminates CGO dependencies. This removes any requirement for external GCC compilers on the host system.
- **Schema-on-Startup Integration:** The application leverages GORM's `AutoMigrate` routines at boot time. This ensures database files and their relational tables are provisioned automatically upon initialization on any host machine.

6 Adopted Technologies

The e-commerce software stack leverages modern technologies designed for performance, modularity, and rapid development:

- **Go (Golang):** Chosen as the primary programming language due to its high-performance execution, excellent native HTTP library support, and first-class concurrency primitives (goroutines and channels).
- **gRPC & Protocol Buffers (Proto3):** Used as the high-performance RPC framework. Protocol Buffers enable strong api typing, explicit interface definitions, and compact binary serialization that outperforms traditional JSON-over-HTTP communications.
- **GORM & SQLite:** GORM acts as the Object-Relational Mapper, providing clean programmatic interactions with the underlying SQLite databases, abstracting raw SQL queries into safe, structured Go code.
- **Gorilla Sessions:** Utilized exclusively to manage signed, secure server-side session cookies to authenticate web users safely.

7 Testing

To guarantee software reliability and protect the codebase against regression defects, each microservice includes an isolated unit testing suite located inside its respective `internal/tests` directory.

The test design heavily emphasizes **reproducibility** and **isolation**. Since running tests leaves databases in a mutated, unpredictable state, running tests iteratively requires an automated teardown mechanism. The testing workflow is bound to the system's global orchestration scripts: prior to launching tests, database clearing commands are issued to wipe data. Upon boot, seed functions re-populate deterministic datasets. This approach establishes a clean, predictable state machine for every test run, allowing developers to execute assertions against stable boundaries.

7.1 Isolated In-Memory Unit Testing

To guarantee that testing passes quickly and remains entirely isolated from persistent storage state, individual repository test suites swap out file-based databases for volatile, in-memory instances:

```
db, err := gorm.Open(sqlite.Open(":memory:"), &gorm.Config{})
```

This approach provides an excellent, side-effect-free environment for executing deterministic assertions against repository operations. Note that executing the full integration test suites on Windows environments exhibits significantly slower processing times compared to native execution on Linux systems due to operating system differences in process management and file system synchronization performance.

8 Deployment and Execution Guide

This section describes the workflow required to compile, build, and execute the entire distributed e-commerce platform.

8.1 Prerequisites and Requirements

Before executing the orchestration commands, the following environment tools must be installed on the host system:

- **Go SDK (v1.22+ recommended):** To compile the microservices, tidy dependencies, and execute the orchestrator script.
- **Protocol Buffers Compiler (`protoc`):** Along with the plugins `protoc-gen-go` and `protoc-gen-go-grpc` to compile the gRPC interface files.

8.2 Network Configuration and Port Allocation

To ensure that the distributed e-commerce platform initializes and communicates correctly, the host machine must have the network ports in the range of `8080` to `8085` completely free and unblocked by other active processes or local firewalls.

Each port in this block is strictly dedicated to a specific microservice within the architecture. If any of these ports are already in use, the orchestrator (`runner.go`) will fail to bind the socket for the respective service, causing a startup crash.

The specific port allocation and functional roles are defined as follows:

Service	**Port**
`Web Server`	8080
`Authentification Service`	8081
`Cart Service`	8082
`Catalog Service`	8083
`Order Service`	8084
`Payment Service`	8085

8.3 The Go-Based Initiator (`runner.go`)

To guarantee a seamless, zero-dependency deployment pipeline across different environments, the project features a centralized orchestrator written in pure Go (`runner.go`).

By leveraging the Go standard library, this tool completely replaces the need for platform-specific utilities (such as GNU Make, Bash, or PowerShell). It programmatically queries the host operating system using Go's `runtime.GOOS` and resolves file paths dynamically with the `path/filepath` package. This approach prevents common cross-platform issues ensuring that compilation, execution, cleanups, and signal handling behave identically on Windows, Linux, and macOS.

At the beginning of the runner.go file, the `//go:build ignore` compilation directive has been introduced. This specific comment, known in Go as a build tag, explicitly instructs the compiler to exclude the file from any automatic, global compilation or testing processes executed within the codebase. This precaution is of fundamental importance for the project's code architecture: it prevents any namespace or signature conflicts related to package main and avoids the collision of multiple `main()` entry points that would otherwise occur when compiling the individual microservices. Thanks to this directive, the orchestrator is completely isolated from the production-ready software, ensuring the system treats it solely as a development utility script that is only executed via a manual, targeted invocation of the `go run runner.go` command.

The following table summarizes all the automated routines available through the orchestrator script:

Command / Flag	Operational Description
`go run runner.go`	The default routine. It compiles all microservices on-the-fly and executes them concurrently in the same terminal, streaming unified, prefixed logs.
`go run runner.go -test`	Performs unit-tests on the single micro-services.
`go run runner.go -proto`	Invokes the system's local `protoc` compiler to regenerate all Go and gRPC files from the `.proto` definitions.
`go run runner.go -clean-proto`	Recursively traverses the directory structure to safely delete all autogenerated Protobuf files (`*.pb.go` and `_grpc.pb.go`).
`go run runner.go -clean-db`	Scans the workspace and purges all persistent SQLite database (`.db`) files to easily reset the platform state.
`go run runner.go -clean`	Performs a targeted cleanup by deleting any residual compiled executables from the respective service directories.

After executing the `runner.go` utility script, the functionality of the developed e-commerce platform can be tested by entering `http://localhost:8080/welcome` into the address bar of any web browser. This action loads the website's homepage, allowing users to navigate the platform freely.